

An End-to-End ECG Monitoring System: Bluetooth Streaming, Mobile Display, and Automatic Heartbeat Classification

Author Name

Department of Electrical and Computer Engineering

University Name

City, Country

email@university.edu

Abstract—This report describes a complete ECG monitoring system built from five parts. A Raspberry Pi reads a recording from the PhysioNet MIT-BIH Arrhythmia Database, resamples it to 250 Hz, and sends it over Bluetooth to an Android phone. The phone receives the signal and draws it on screen in real time. It then sends the signal over Wi-Fi to a server, which finds each heartbeat, sorts it into normal or abnormal, and sends the answer back. The same analysis was also written in Java and run on the phone itself, so the system still works without a network. Across 22 patients not used during development, heartbeat detection reached 98.0% sensitivity and 99.8% precision. A decision tree classifier reached 77.2% overall accuracy and correctly identified 76.0% of ventricular ectopic beats. An automatic test confirms that the phone and the server produce identical results from the same signal.

Index Terms—electrocardiography, QRS detection, Pan-Tompkins, arrhythmia classification, Bluetooth, Android, mobile health

I. INTRODUCTION

A long ECG recording contains far more heartbeats than a doctor can check by hand. A single day of monitoring holds around 100,000 of them. Software that finds the abnormal beats automatically is therefore essential if monitoring is going to be useful outside a hospital.

This project builds the full chain such a system needs. The assignment set five tasks, and this report follows them in order:

- 1) send an ECG signal at 250 Hz over Bluetooth;
- 2) receive that signal on a phone;
- 3) draw the signal on the phone screen;
- 4) send the signal onward to a server over Wi-Fi;
- 5) run signal processing and a classifier on the server, and send the results back to the phone.

An optional extra task was to run the signal processing and the classifier on the phone instead of the server. This was also completed. Because the analysis then exists twice, in Python and in Java, an automatic test was written to check that the two versions agree exactly.

Section II gives an overview of the system. Sections III to VI cover the five tasks in turn. Section VII covers the on-phone version, Section VIII gives the results, and Section IX discusses what the system does not do well.

II. SYSTEM OVERVIEW

Fig. 1 shows how the parts fit together. A Raspberry Pi acts as the sensor. An Android phone receives, displays and optionally analyses the signal. A laptop acts as the server. All three sit on the same Wi-Fi network.

III. PART 1: SENDING THE ECG OVER BLUETOOTH

A. Getting the signal

Real ECG recordings were taken from the MIT-BIH Arrhythmia Database [2] on PhysioNet [3]. Each recording comes with labels written by cardiologists saying what type every heartbeat is. Those labels are what make it possible to measure how well the classifier works.

The database is recorded at 360 Hz, but the assignment asks for 250 Hz. The ratio 250/360 simplifies to 25/36, so the signal is resampled by a factor of 25 up and 36 down using a polyphase filter, which filters the signal before throwing samples away.

It is worth being clear about why that filter matters, because the point is easy to overstate. Without it, frequencies between 125 Hz and 180 Hz fold down into the 70–125 Hz range, so a 150 Hz muscle artefact would reappear as a false 100 Hz signal, as Fig. 2 shows. This folding cannot reach the 5–15 Hz band used to find heartbeats, because that would need energy at 235–245 Hz, which a 360 Hz recording cannot contain. Beat detection is therefore safe either way. What the filter protects is the shape of the waveform shown on screen.

B. The packet format

Bluetooth Serial Port Profile sends a plain stream of bytes with no markers showing where one message ends and the next begins, so the program adds its own structure:

```
0xA5 0x5A | seq | count | 25 samples | checksum
```

Each packet holds 25 samples, which is 100 ms of signal, so ten packets are sent every second. Sending one sample per packet would waste more space on headers than on the data itself.

Three things can go wrong on a wireless link: the phone can connect halfway through a packet, a byte can be corrupted, and

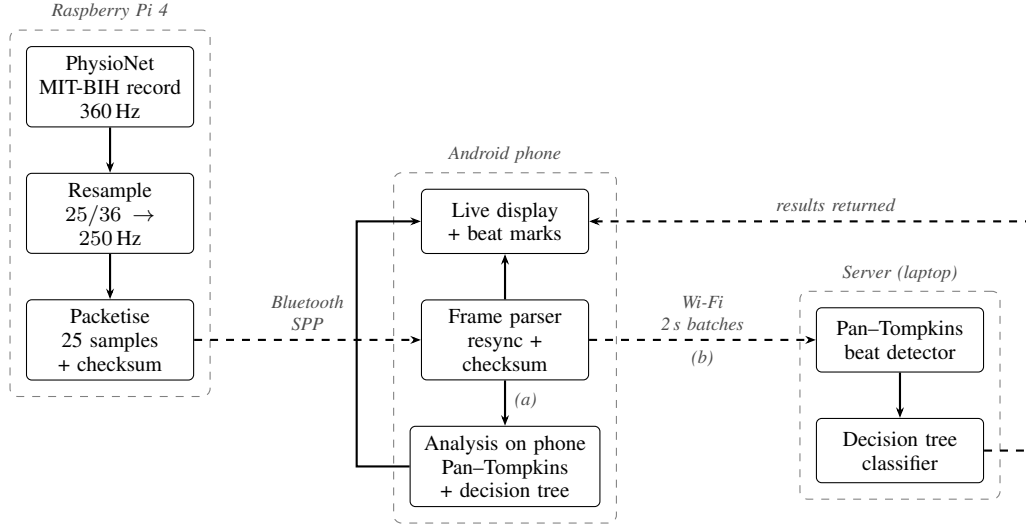


Fig. 1. The complete system. Solid arrows are processing steps inside one device; dashed arrows are wireless links. The beat analysis exists twice and the phone uses one path at a time: (a) everything runs on the phone and no network is needed, or (b) the signal is sent to the server and the labels come back. The two paths run identical algorithms, one written in Java and one in Python, and an automatic test confirms they agree exactly.

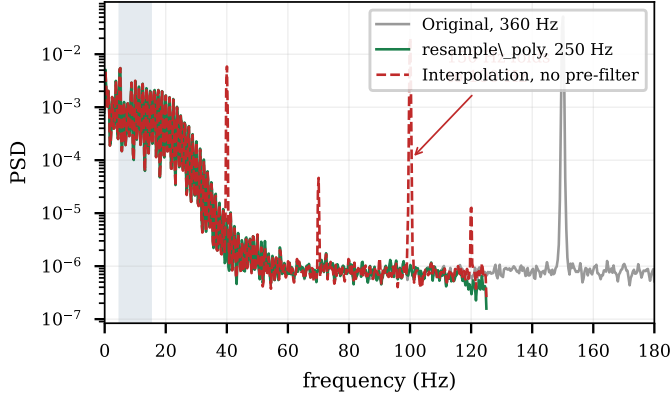


Fig. 2. Frequency content before and after resampling from 360 Hz to 250 Hz. Without a filter a 150 Hz component reappears at 100 Hz. The shaded band is where the beat detector works.

data arrives in chunks of random size. The receiver handles all three by searching for the two marker bytes, checking the checksum, and moving forward one byte at a time whenever something does not match. This loses one packet instead of the whole connection.

One detail proved important during testing. The receiver must check that the length byte is sensible *before* using it. A corrupted length claiming that 512 samples follow made the receiver wait forever for bytes that would never arrive, freezing the link permanently rather than losing a single packet.

IV. PARTS 2 AND 3: RECEIVING AND DISPLAYING ON THE PHONE

The Android app connects to the Raspberry Pi over Bluetooth and reads the byte stream. Reading, decoding and filtering all happen on a background thread. Only short text

updates are sent to the screen thread, so the display stays smooth while data arrives continuously.

The waveform is drawn the way a hospital monitor draws it. New samples are written into a circular buffer and a small gap is cleared just ahead of the writing position, which gives a sweeping trace. The simpler approach of shifting the whole array left on every frame was avoided because it copies the entire buffer every frame and makes the trace flicker.

The screen is redrawn about 30 times per second rather than every time a packet arrives. The display cannot show more than this, and redrawing more often only wastes battery. Where the screen size allows, the grid follows the standard ECG paper scale of 25 mm/s and 10 mm/mV, so the trace looks familiar.

V. PART 4: SENDING THE SIGNAL TO THE SERVER

The phone collects two seconds of signal (500 samples) before sending anything. Sending each 25-sample packet immediately would mean ten network requests every second, and each request carries more header information than actual data, so the battery cost would be high.

The waiting queue has a fixed size. If the network cannot keep up, the oldest batch is thrown away rather than stored. For a live monitor old ECG data is of little use, and a queue with no limit simply grows until the phone runs out of memory and closes the app.

VI. PART 5: SERVER PROCESSING AND CLASSIFICATION

A. Finding the heartbeats

Heartbeats are found using the Pan-Tompkins method [1], written to process one sample at a time so it can run on a live stream. The stages are shown in Fig. 4:

- 1) a 5–15 Hz bandpass filter, which keeps the sharp part of the heartbeat and removes baseline drift and muscle noise;



Fig. 3. The app showing a live ECG. Marks above the trace are coloured by beat type: green for normal, amber for supraventricular, red for ventricular.

- 2) a derivative, which responds to how steeply the signal rises;
- 3) squaring, which makes everything positive and exaggerates large slopes;
- 4) a 150 ms moving average, which turns each beat into a single smooth bump.

Bumps are then accepted as heartbeats using thresholds that adapt to the signal, with a 200 ms blocking period so one beat cannot be counted twice, and a check that rejects T waves by comparing how steeply they rise.

B. Locating the exact beat position

Finding the exact position of each beat mattered more than expected, because every shape measurement is taken relative to it.

The first version searched nearby for the largest point of the filtered signal. This failed because a filtered heartbeat swings both up and down, so the search landed on the upward peak for one beat and the downward peak for the next.

The second version searched for whichever position best matched the patient's own normal beat. This was worse, and in a way that was hard to see. Because it kept the best possible match, the similarity score became a best case rather than a fair measurement. On record 233, beats that cardiologists had marked as ventricular scored between 0.83 and 0.98 similarity against the patient's normal beat. The test for "different shape" therefore never triggered, and two-thirds of abnormal beats were missed.

The final version uses the centre of energy of the beat,

$$c = \frac{\sum_k f^2[k] k}{\sum_k f^2[k]}, \quad (1)$$

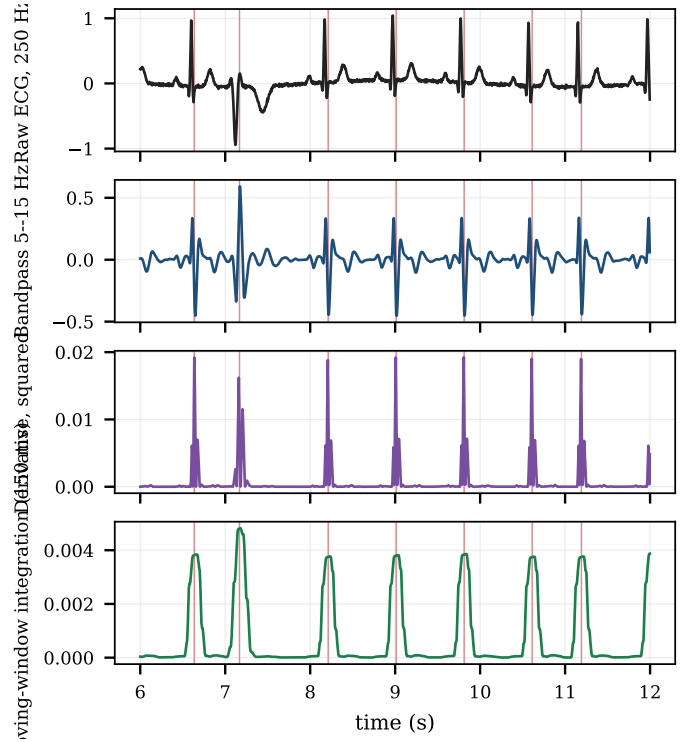


Fig. 4. The four stages of the beat detector over six seconds of ECG. Vertical lines mark where beats were found. Each stage makes the heartbeat stand out more clearly from everything else.

TABLE I
MEASUREMENTS TAKEN FOR EACH HEARTBEAT

Name	Meaning	Tells us
rr_prev	gap before the beat (s)	heart rate
rr_next	gap after the beat (s)	pause length
rr_ratio	gap before / average	how early
rr_next_ratio	gap after / average	pause after
width_ratio	width / normal width	how wide
amp_ratio	height / normal height	signal quality
corr	similarity to normal	shape
rr_asym	change in gap	rhythm pattern
area_ratio	area / normal area	size of the beat

where $f[k]$ is the filtered signal. This is a plain physical definition, the middle of the beat, which means the same thing for a narrow normal beat and a wide abnormal one. It does not look at the template at all, so the similarity score measured afterwards is honest.

C. Describing each beat

Nine measurements are taken for every beat, listed in Table I. They are written as *ratios* against the patient's own normal beats rather than as fixed numbers of millivolts or milliseconds. Fixed limits do not transfer between patients or between electrode positions; ratios largely do.

Each beat is classified one beat late, so that the gap *after* it is known. This delay is deliberate. The standard way to tell a ventricular beat from an atrial one is the pause that follows it. A ventricular beat does not reset the heart's natural pacemaker,

so the next normal beat arrives on schedule and the pause is long. An atrial beat does reset it, so the pause is shorter.

Fig. 5 shows how well these measurements separate the beat types on real data.

D. Choosing the classifier

Two classifiers were compared. The first uses fixed rules written by hand from textbook criteria and needs no training data. The second is a decision tree fitted to the labelled data.

Fixed rules turned out to be hard to set correctly. Thresholds were first chosen using a synthetic test signal, and twice they behaved differently on real recordings. In one case a threshold was set from the list of wrongly classified beats, which are by definition the unusual ones, and the conclusion drawn from them did not hold for the rest of the data. Fitting the thresholds to the data avoided this.

A decision tree of depth 3 was chosen. Deeper trees scored *worse* on patients they had not seen, because they began learning the habits of individual patients rather than general rules. A shallow tree is also small enough to copy onto the phone as a few comparisons, and simple enough to read, so the rules it learned can be checked against medical expectation. Those rules were sensible: it splits first on how early the beat is, then uses *how much* earlier to separate ventricular from atrial beats, and falls back on shape for beats that are not early.

Logistic regression was also tried and performed noticeably worse (Table IV), which suggests the boundary between beat types is not a straight line in these measurements.

VII. OPTIONAL TASK: RUNNING EVERYTHING ON THE PHONE

The detector and the classifier were rewritten in Java and run on the phone. The cost is about 40 arithmetic operations per sample, which at 250 Hz is far below what any modern phone can do. With this enabled the phone needs no network at all.

Keeping the same method written twice, once in Python and once in Java, creates an obvious risk: the two could quietly drift apart and give different answers for the same patient. An automatic test was therefore written. It compiles the phone's code with a normal Java compiler, runs both versions over the same signal, and compares every beat position, every measurement and every label. It reports agreement to six decimal places, and it is run after any change to either version.

VIII. RESULTS

A. The wireless link

The packet decoder was tested against the three failure types described in Section III. Results are in Table II. The output was identical no matter how the data was broken into chunks, and any single corrupted byte cost exactly one packet.

A full test streaming 12 seconds of ECG from the transmitter to the receiving code delivered 120 packets and 3000 samples with nothing lost, which is exactly the expected rate of 250 samples per second.

TABLE II
BEHAVIOUR OF THE PACKET DECODER UNDER FAULTS

Test	Result
Chunks of 1, 7, 56, 4096 bytes	identical output every time
Corrupted marker bytes	1 packet lost, then recovered
Corrupted length byte	1 packet lost, then recovered
Corrupted data or checksum	1 packet lost, then recovered
Joining partway through	recovered at the next packet

TABLE III
BEAT DETECTION ON 22 UNSEEN PATIENTS

Beats marked by cardiologists	49,712
Beats found by the detector	48,834
Correctly matched	48,740
Sensitivity (beats found)	98.04%
Precision (found beats that were real)	99.81%

TABLE IV
CLASSIFIER COMPARISON, 22,781 BEATS, PATIENT-DISJOINT TESTING

	Fixed rules	Logistic	Tree
Overall accuracy	72.2%	63.4%	77.2%
Normal, found	78.3%	73.1%	87.6%
Normal, correct	95.8%	90.7%	86.5%
Ventricular, found	80.1%	46.0%	76.0%
Ventricular, correct	39.0%	37.1%	67.3%
Supraventricular, found	15.0%	8.7%	2.3%

B. Finding heartbeats

Beat detection was measured on 22 recordings that were not used during development, following the standard patient split of de Chazal *et al.* [4]. A beat counted as found if it lay within 150 ms of a cardiologist's mark.

Nearly all recordings were close to perfect. The exception was record 228, where only about half the beats were found. That recording is known for heavy noise and a wandering baseline, and the system has no check for poor signal quality.

C. Classifying heartbeats

The classifiers were compared on 22,781 labelled beats from ten recordings using leave-one-record-out testing. Every recording is tested with a classifier trained only on the other nine, so no classifier is ever tested on a patient it has already seen.

The decision tree is the best overall. The most useful difference is how often it is right when it reports a ventricular beat: 67.3%, against 39.0% for the fixed rules. A monitor that raises a false alarm two times out of three is one that staff will start to ignore, so this matters more than the headline accuracy figure. Table V gives the tree's results in full.

D. Speed

The server analysed a two-second batch of ECG in about 1.8 ms on average. That is roughly a thousand times faster than real time, so one server could handle a large number of

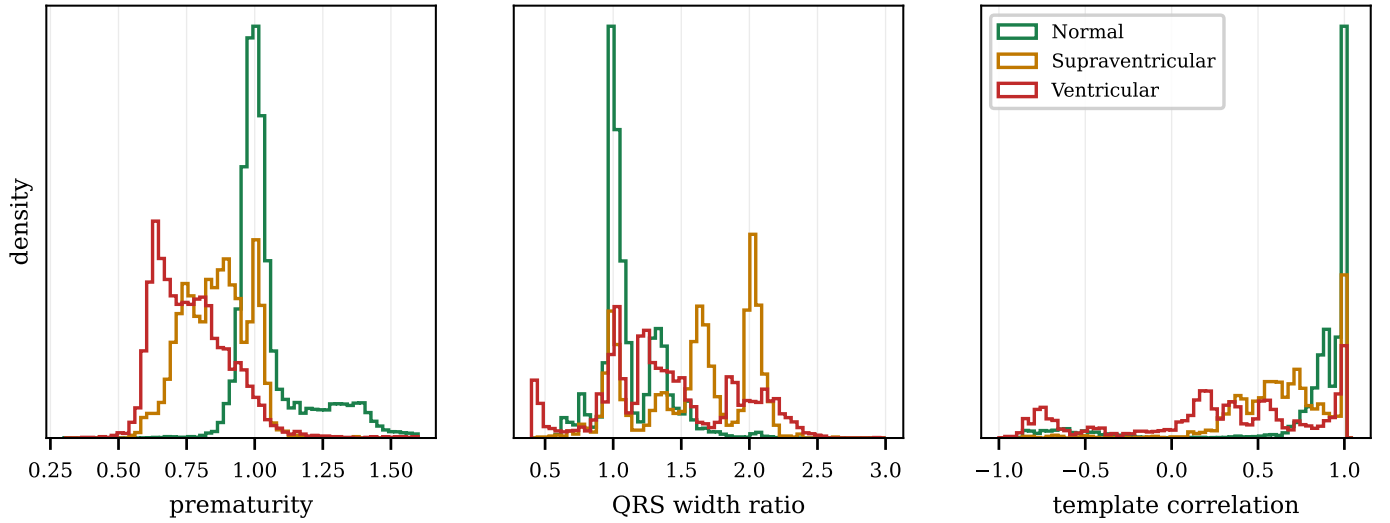


Fig. 5. How three of the measurements separate the beat types, using 22,781 real heartbeats from MIT-BIH with cardiologist labels. Ventricular beats are earlier, wider and less similar in shape. The overlap between the curves is why no single measurement is enough on its own.

```

if how early (gap before / average)  $\leq$  0.895:
    if gap before beat (s)  $\leq$  0.638:
        if QRS width (s)  $\leq$  0.053:
            Ventricular (68% confident)
        else:
            Ventricular (97% confident)
    else:
        if gap before beat (s)  $\leq$  0.792:
            Supraventricular (89% confident)
        else:
            Normal (95% confident)
else:
    if QRS width / normal width  $\leq$  1.594:
        if area / normal area  $\leq$  1.729:
            Normal (96% confident)
        else:
            Ventricular (49% confident)
    else:
        if shape similarity to normal  $\leq$  0.258:
            Ventricular (89% confident)
        else:
            Normal (53% confident)

```

Fig. 6. The decision tree learned from the training patients. Its depth is capped at 3 so that it stays readable and can be checked against clinical expectation: it splits first on how early the beat is, then on how much earlier, and falls back on shape for beats that are not early.

TABLE V
DECISION TREE RESULTS IN DETAIL (ROWS ARE THE TRUE TYPE)

	Normal	Supra.	Vent.	Other
Normal	14,572	1,342	685	32
Supraventricular	1,068	43	725	6
Ventricular	870	71	2,981	0
Other	345	1	40	0



Fig. 8. The server dashboard receiving live data from the phone and showing the classified beats being sent back.



Fig. 7. Output of the automatic test comparing the Python and Java versions of the analysis.

phones at once. On the phone the cost per sample is small enough that the app stayed responsive throughout.

IX. LIMITATIONS

Supraventricular beats are not detected well. Only 2.3% are found. These beats travel the normal path through the heart, so they look almost identical to normal beats. The real clue is the P wave in front of them, and none of the measurements in this system look at the P wave. This is an honest limit of the chosen measurements, not a tuning problem.



Fig. 9. The test setup. The Raspberry Pi stands in for a wearable sensor by streaming a recorded ECG over Bluetooth.

There is no signal quality check. If an electrode comes loose the detector will happily find “beats” in the resulting noise. Record 228 shows this clearly. A real device would need to detect poor contact and stop reporting.

The signal is a recording, not a live patient. The system has never faced movement artefacts, sweat, or a loose electrode, which are usually the main sources of error in real monitoring.

The “Other” group performs poorly. It mixes fusion beats, paced beats and unclassifiable ones together, and none of them are identified.

This is not a medical device. It was tested on a database, not validated clinically, and must not be used to make decisions about anyone’s health.

X. CONCLUSION

All five parts of the assignment were completed, along with the optional task of running the analysis on the phone. The system reads a real ECG, sends it over Bluetooth, displays it live, forwards it over Wi-Fi, and classifies each beat either on the server or on the phone. Beat detection reached 98.0% sensitivity and 99.8% precision on 22 unseen patients, and the decision tree classifier reached 77.2% accuracy with 76.0% of ventricular beats correctly found.

The most useful lesson was about testing. Every serious problem in this project—the frozen link caused by a bad length byte, the similarity score that flattered abnormal beats, and thresholds that suited a synthetic test signal but not real patients—was found by measuring, not by reading the code. Two of them looked perfectly correct until real data disagreed. The automatic test comparing the phone and server versions exists for exactly that reason.

ACKNOWLEDGMENT

The authors thank the PhysioNet project for providing free access to the MIT-BIH Arrhythmia Database.

REFERENCES

- [1] J. Pan and W. J. Tompkins, “A real-time QRS detection algorithm,” *IEEE Trans. Biomed. Eng.*, vol. BME-32, no. 3, pp. 230–236, Mar. 1985.
- [2] G. B. Moody and R. G. Mark, “The impact of the MIT-BIH Arrhythmia Database,” *IEEE Eng. Med. Biol. Mag.*, vol. 20, no. 3, pp. 45–50, May 2001.
- [3] A. L. Goldberger *et al.*, “PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals,” *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.
- [4] P. de Chazal, M. O’Dwyer, and R. B. Reilly, “Automatic classification of heartbeats using ECG morphology and heartbeat interval features,” *IEEE Trans. Biomed. Eng.*, vol. 51, no. 7, pp. 1196–1206, Jul. 2004.
- [5] *Testing and Reporting Performance Results of Cardiac Rhythm and ST Segment Measurement Algorithms*, ANSI/AAMI Standard EC57, 2012.
- [6] E. J. S. Luz, W. R. Schwartz, G. Cámara-Chávez, and D. Menotti, “ECG-based heartbeat classification for arrhythmia detection: A survey,” *Comput. Methods Programs Biomed.*, vol. 127, pp. 144–164, Apr. 2016.